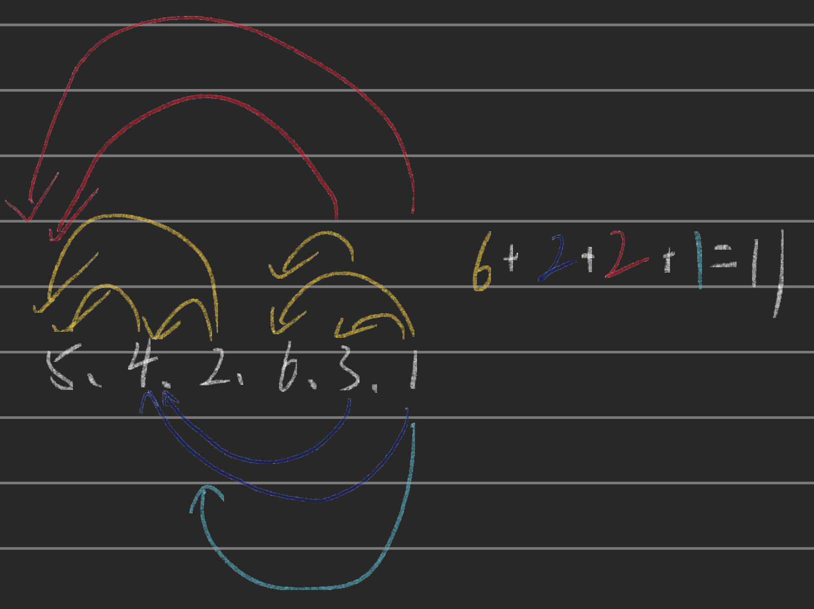


```
#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;

int n;
vector<int> a;
vector<int> b;
vector<int> bit;

int lowbit(int x) {
  return x & -x;
}

void add(int i, int v) {
  while (i <= n) {
    bit[i] += v;
    i += lowbit(i);
  }
}

int sum(int i) {
  int s = 0;
  while (i > 0) {
    s += bit[i];
    i -= lowbit(i);
  }
  return s;
}

int main() {
  ios::sync_with_stdio(false);
  cin.tie(nullptr);

  cin >> n;
  a.resize(n);
  for (int i = 0; i < n; i++) {
    cin >> a[i];
  }

  b = a;
  sort(b.begin(), b.end());
  b.erase(unique(b.begin(), b.end()), b.end());

  n = b.size();         // ← 关键：n 改成离散化后的大小
  bit.assign(n + 1, 0);
  long long ans = 0;
  for (int i = a.size() - 1; i >= 0; i--) {  // ← 注意：循环用 a.size()
    int rank = lower_bound(b.begin(), b.end(), a[i]) - b.begin() + 1;
    ans += sum(rank - 1);
    add(rank, 1);
  }
  cout << ans;
  return 0;
}
```

更新 add(i,v)对下标i做单点更新，并沿lowbit路径向上覆盖i的所有节点，使各节点维护的区间和增加v

查询 sum(i): 前i个值的总和

↓

并非bit, b, a数组中前i个数的总和，
而是一个被隐藏掉的cnt[]数组中的总和

cnt[i]：记录排名为i的数字出现过多少次

cin >> a[i]; 将题目原本涉及的所有坐标全部加进数组a中

b = a; →将数组a复制一份为b

}将数组b排序后去重

将n改为数组b当前的键数

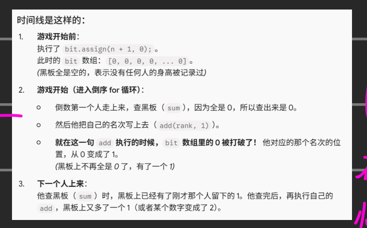
将树状数组重制为n+1个0

↱针对逆序对，倒序遍历

当前排位
(也就是排名+1

→在已经排好序的区间里，
找到第一个≥a[i]的位置，
返回的是迭代器

离散化的概念：
排序+去重+映射
核心思想：
将坐标范围极大，
但稀疏的数据映
射到紧凑的整数区间


时间线是这样的：

1. **游戏开始前：**
   执行了 `bit.assign(n + 1, 0);`。
   此时的 `bit` 数组：`[0, 0, 0, 0, ... 0]`。
   (黑板全是空的，表示没有任何人的身高被记录过)
2. **游戏开始（进入倒序 for 循环）：**
   - 倒数第一个人走上来，查黑板（`sum`），因为全是 0，所以查出来是 0。
   - 然后他把自己的名次写上去（`add(rank, 1)`）。
   - **就在这一句 `add` 执行的时候，`bit` 数组里的 0 被打破了！** 他对应的那个名次的位置，从 0 变成了 1。
     (黑板上不再全是 0 了，有了一个 1)
3. **下一个人上来：**
   他查黑板（`sum`）时，黑板上已经有了刚才那个人留下的 1。他查完后，再执行自己的 `add`，黑板上又多了一个 1（或者某个数字变成了 2）。

lower_bound 的任务：
在已经排好序且去重的数组中，
快速找到原数组元素a[i]对应的位置(也就是排名)

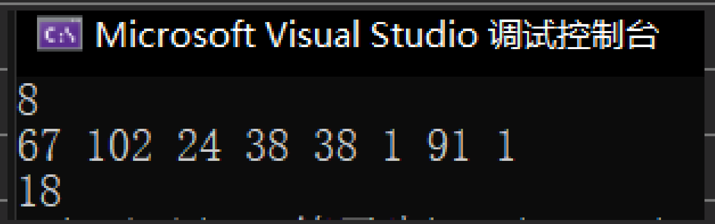
lower_bound (b.begin(), b.end(), a[i])

b.begin()：开始查找的位置

b.end()：结束查找的位置

a[i]：所要查找的值

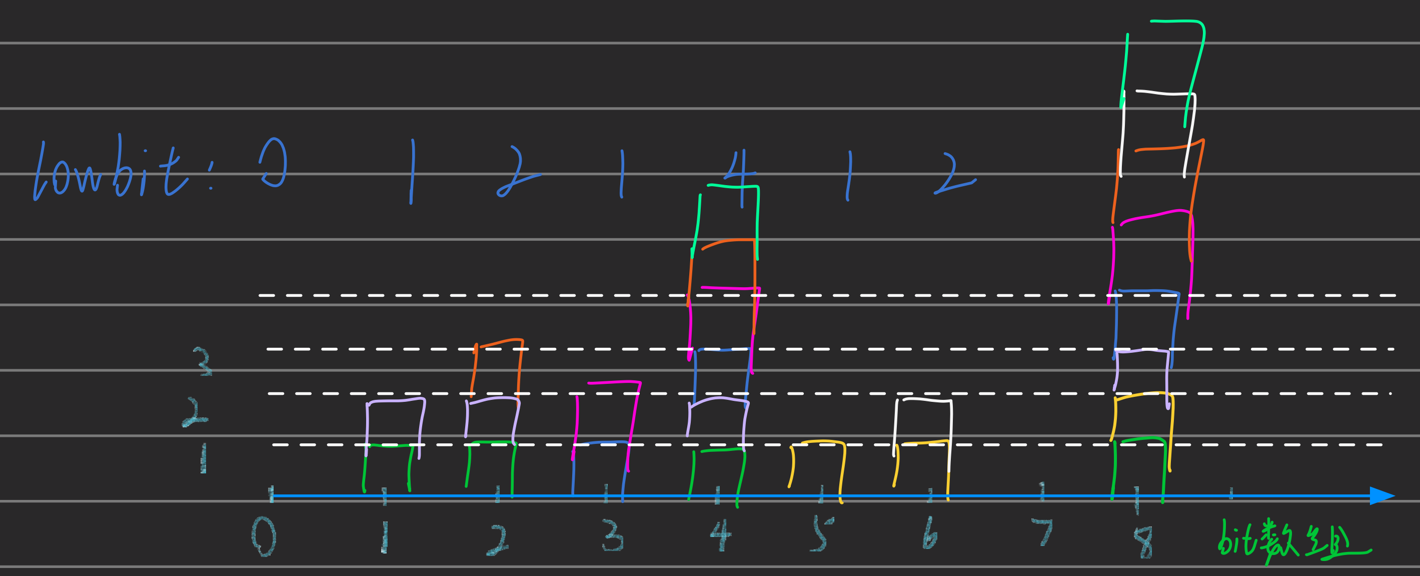
因为bit中都是0，
所以不论第一个输进去的
是什么数，sum都是0
↓
然后进行第一次add，
对bit中底标rank和rank的上层的数全部加1
↓
然后进行第二次sum：情况A：如果倒数第二个数，比倒数第一个数大：
90往右看，发现了一个比自己小的数，sum=1，ans += 1；
情况B：如果倒数第二个数，比倒数第一个数小：
sum=0，ans +=0，

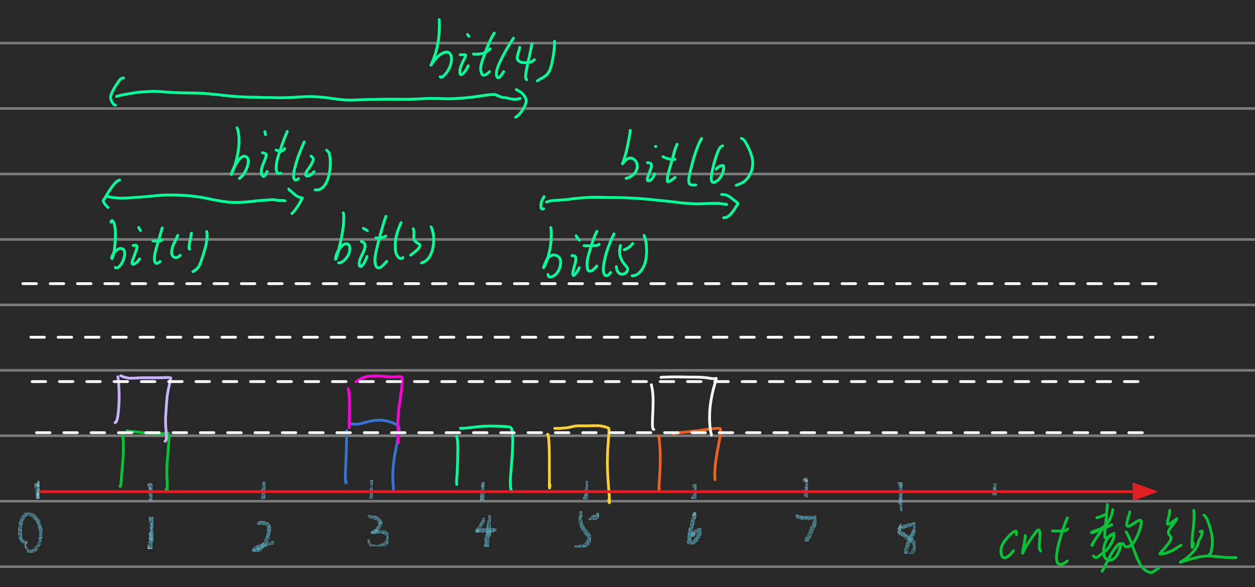


```
8
67 102 24 38 38 1 91 1
18
```

for(a.size-1, i>=0, i--)
↓
倒序遍历

原数据：67、102、24、38、38、1、91、1
经排序、去重、映射后得到的数据：1、2、3、4、5、6





	step1	step2.	step.3	step.4	step.5	step.6	step.7	step.8
原	1	91	1	38	38	24	102	67
rank	1	5	1	3	3	2	6	4
sum	0	1	0	2	2	2	6	5
add	(1,1)	(5,1)	(1,1)	(3,1)	(3,1)	(2,1)	(6,1)	(4,1)

	bit(1)+1;
	bit(2)+1;
	bit(4)+1;
	bit(8)+1;
		bit(5)+1
		bit(6)+1
		bit(8)+1
			bit(1)+1
			bit(2)+1
			bit(4)+1
			bit(8)+1
				bit(3)+1
				bit(4)+1
				bit(8)+1
					bit(3)+1
					bit(4)+1
					bit(8)+1
						bit(2)+1
						bit(4)+1
						bit(8)+1
							bit(6)+1
							bit(8)+1
								bit(4)+1
								bit(8)+1

ans：	0	1	1	3	5	7	13	18